Computer Systems, Data Structures & Data Management

Markos Vassilis
Mediterranean College, Spring 2024-2025

Week 18

O(n) Revisited

Warm up

Right or wrong?

- f(n) = n^2 + 6 * n is O(n).
  - Wrong!
- f(n) = n^2 + 6 * n is O(n^2).
  - Right!
- f(n) = n^2 + 6 * n is O(n^4).
  - Right!
- f(n) = n^2 + 6 * n is O(2^n).
  - Right!
- f(n) = n^2 + 6 * n is O(log(n)).
  - Wrong!

(Reminder) A rule of thumb

Some of the most popular complexity classes, ordered in increasing complexity (i.e., every function f(n) is O(g(n)) for any g(n) below f(n)):

- f(n) = 1
- f(n) = log(n)
- f(n) = sqrt(n)
- f(n) = n
- f(n) = nlog(n)
- f(n) = n^2
- f(n) = n^3
- f(n) = 2^n
- f(n) = n!
- f(n) = n^n

Exercise

What is the (asymptotic) worst-case computational complexity of the following algorithm for an input list, a, of size n?

```
for each x in a:
    if x is odd:
        print(“Odd”)
    else:
        print(“Even”)
```

Answer: O(n), since we access each of its elements just once.

Exercise

What is the (asymptotic) worst-case computational complexity of the following algorithm for an input list, a, of size n?

```
read x from user
for each y in a:
    if x == y:
        print(“Found!”)
        return
print(“Not found!”)
return
```

Answer: O(n), since we access each of its elements just once in the worst case, which is that the element we have to look for (x) is either last or does not exist in a.

Sorting (Revisited)

Sorting

- It is nice to have things in place. Consider the following array:

```
arr = [3, 6, 1, 7, 9, 0, 2];
```

- Evidently, `arr` is not sorted in any meaningful way. How can we put thing in increasing order (in a systematic way).
- [Take some time to devise a sorting algorithm that can work for arbitrary size arrays, not just for `arr`.]

Bubbles. I want more bubbles.

3	6	1	7	9	0	2

$3 < 6 \rightarrow$ **Things in place!**

3	6	1	7	9	0	2

$6 > 1 \rightarrow$ **Things not in place!**

3	1	6	7	9	0	2

$1 < 6 \rightarrow$ **Swap!**

3	1	6	7	9	0	2

$6 < 7 \rightarrow$ **Things in place!**

Bubbles. I want more bubbles.

3	1	6	7	9	0	2

$7 < 9 \rightarrow$ **Things in place!**

3	1	6	7	9	0	2

$9 > 0 \rightarrow$ **Things not in place!**

3	1	6	7	0	9	2

$0 < 9 \rightarrow$ **Swap!**

3	1	6	7	0	9	2

$9 > 2 \rightarrow$ **Things not in place!**

Bubbles. I want more bubbles.

3	1	6	7	0	2	9

$2 < 9 \rightarrow$ **Swap!**

3	1	6	7	0	2	9

$3 > 1 \rightarrow$ **Things not in place!**

1	3	6	7	0	2	9

$1 < 3 \rightarrow$ **Swap!**

1	3	6	7	0	2	9

$3 < 6 \rightarrow$ **Things in place!**

Bubbles. I want more bubbles.

1	3	6	7	0	2	9

$6 < 7 \rightarrow$ **Things in place!**

1	3	6	7	0	2	9

$7 > 0 \rightarrow$ **Things not in place!**

1	3	6	0	7	2	9

$0 < 7 \rightarrow$ **Swap!**

1	3	6	0	7	2	9

$7 > 2 \rightarrow$ **Things not in place!**

Bubbles. I want more bubbles.

1	3	6	0	2	7	9

$2 < 7 \rightarrow$ **Swap!**

1	3	6	0	2	7	9

$1 < 3 \rightarrow$ **Things in place!**

1	3	6	0	2	7	9

$3 < 6 \rightarrow$ **Things in place!**

1	3	6	0	2	7	9

$6 > 0 \rightarrow$ **Things not in place!**

Bubbles. I want more bubbles.

1	3	0	6	2	7	9

$0 < 6 \rightarrow$ **Swap!**

1	3	0	6	2	7	9

$6 > 2 \rightarrow$ **Things not in place!**

1	3	0	2	6	7	9

$2 < 6 \rightarrow$ **Swap!**

1	3	0	2	6	7	9

$1 < 3 \rightarrow$ **Things in place!**

Bubbles. I want more bubbles.

1	3	0	2	6	7	9

$3 > 0 \rightarrow$ **Things not in place!**

1	0	3	2	6	7	9

$0 < 3 \rightarrow$ **Swap!**

1	0	3	2	6	7	9

$3 > 2 \rightarrow$ **Things not in place!**

1	0	2	3	6	7	9

$2 < 3 \rightarrow$ **Swap!**

Bubbles. I want more bubbles.

1	0	2	3	6	7	9

1 > 0 → **Things not in place!**

0	1	2	3	6	7	9

0 < 1 → **Swap!**

0	1	2	3	6	7	9

1 > 2 → **Things in place!**

0	1	2	3	6	7	9

0 < 1 → **Things in place!**

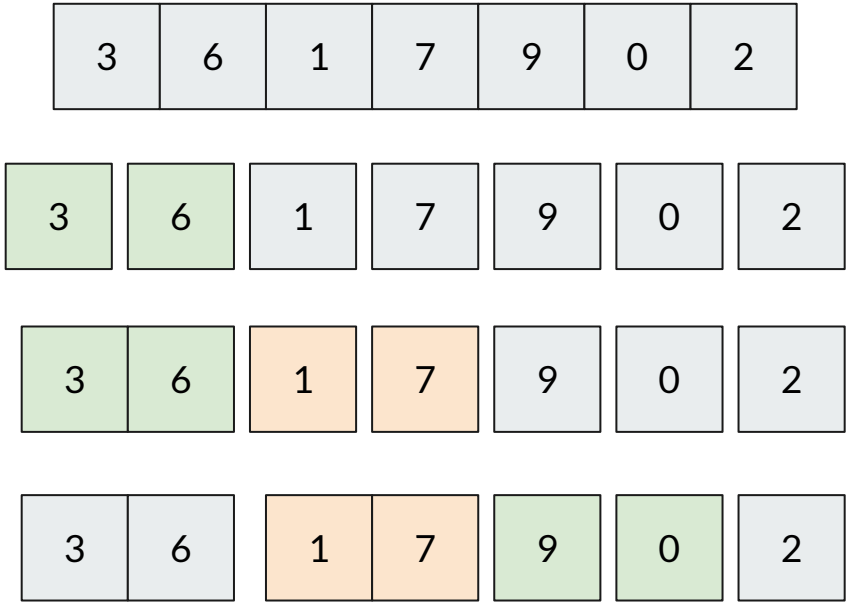
Bubbles. I want more bubbles.

0	1	2	3	6	7	9

Done!

- Really slow, isn't it?
- How "much" would this take to sort one million integers?
- As Obama has said, "Bubble sort is the wrong way to go [when it comes to sorting one million integers]".
- [source]
- Can you think of something faster?

Divide and Conquer



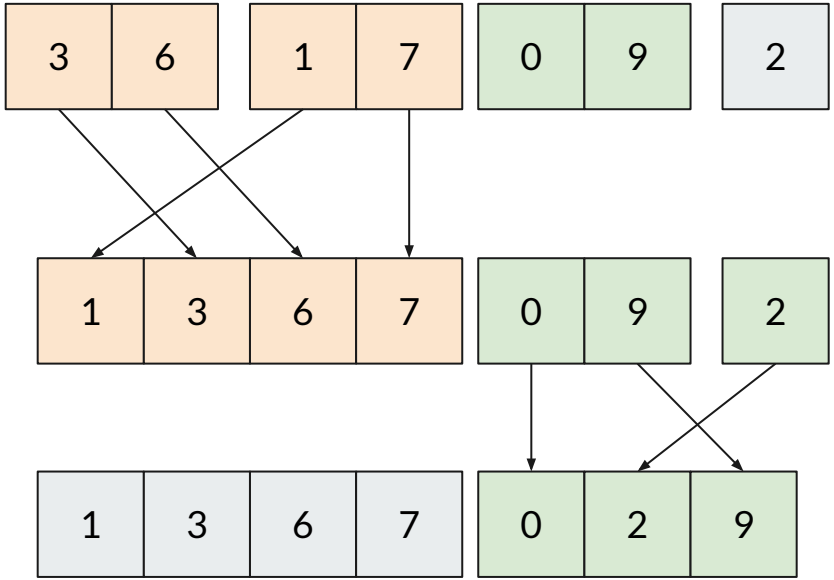
Start with the same list…

Split things to unitary blocks…

Merge the first two, respecting order…

Merge the next two, respecting order…

Divide and Conquer

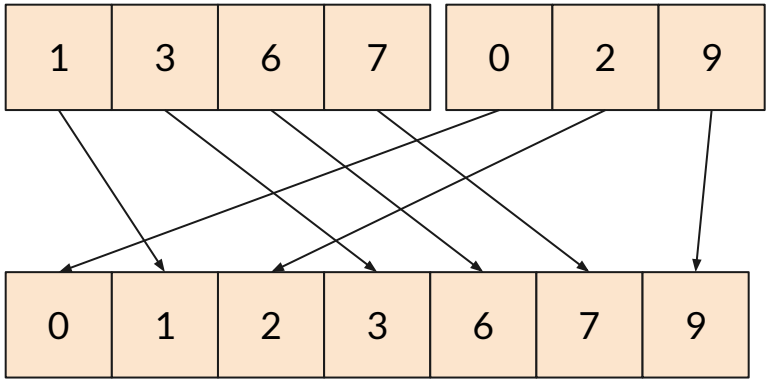


Merge the next two, respecting order…

Merge the next two, respecting order…

Merge the next two, respecting order…

Divide and Conquer

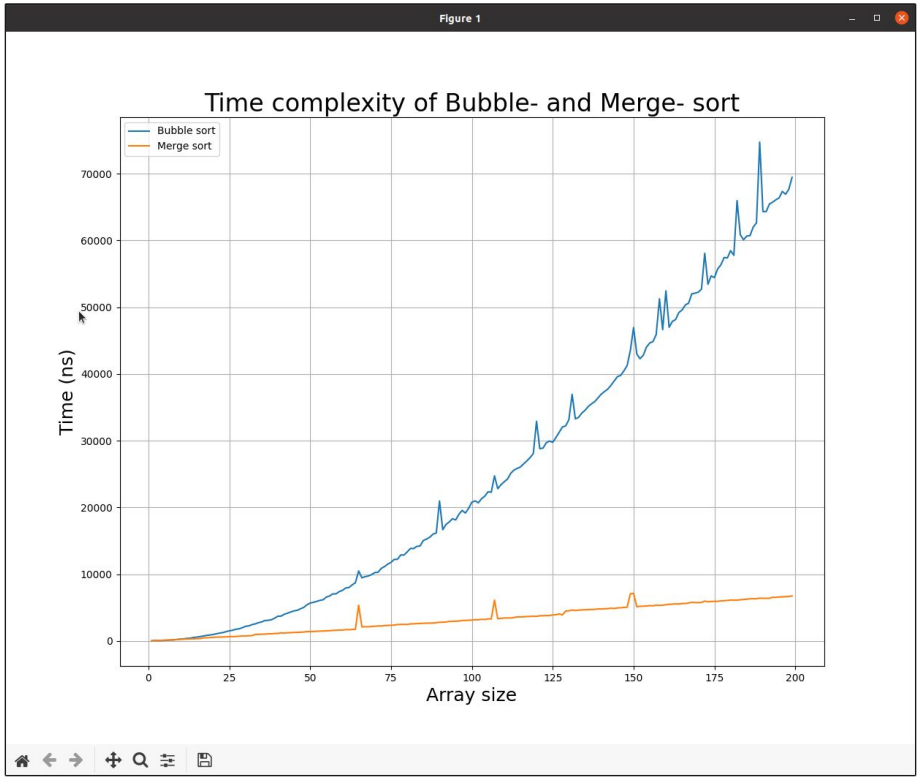


[Repeating from previous slide]

Merge the next two, respecting order…

- Is this any better?
- How much time does it take to merge lists respecting their order?
- Let's compare them…

Hopefully, what you see looks like that…



Bubbles and Complexity

In short, Bubble-sort looks like this:

```
swapped = True, j = 1
while swapped:
    for i = 0,1,..., n - j:
        if arr[i] > arr[i + 1]:
            swap arr[i] and arr[i + 1]
            swapped = True
    j++;
```

arr has been sorted!

What about (worst-case) complexity?

Bubbles and Complexity

In short, Bubble-sort looks like this:

```
swapped = True, j = 1
while swapped: ← This will be executed at most n-1 times
    for i = 0,1,..., n - j: ← This is roughly n iterations
        if arr[i] > arr[i + 1]:
            swap arr[i] and arr[i + 1]
            swapped = True
    j++;
```

Elementary Operation

arr has been sorted!

Worst case complexity: O(n x (n-1)) = O(n^2)

Merging and Complexity

In short, Merge-sort looks like this:

```
n = len(arr)
for i = 0, 1, …, log(len(arr)):
    for j = 0, 1, …, n:
        left = arr[j : j+2^i]
        right = arr[j+2^i : j+2^(i+1)]
        merge(left, right)
    n = n / 2
```

arr has been sorted!

What about (worst-case) complexity?

Merging and Complexity

In short, Merge-sort looks like this:

```
n = len(arr)
for i = 0, 1, …, log(len(arr)): ← This is repeated log(n) times
    for j = 0, 1, …, n: ← Don't fall for this!
        left = arr[j : j+2^i]
        right = arr[j+2^i : j+2^(i+1)]
        merge(left, right) ← len(left) + len(right) EO
    n = n / 2                which means n EO per iteration.
```

arr has been sorted!

Worst case complexity: O(nlog(n))

Why merge log(n) times?

Consider a list of length **8** = 2^3 (just for convenience). Once broken into pieces of length == 1, you have **8** tiny lists of one element each.

- Every time you merge you are left with exactly half (±1) tiny lists (of roughly double size)
- So, after how many "merge runs" will you be left with just one (the initial) list?
- **8** tiny lists → 4 tiny lists → 2 tiny lists → 1 large list.
- At each step you divide by two, so, essentially, you are looking for the number of consecutive divisions by 2 required to reach 1, starting from 8.
- Reading this right to left, we can count the number of multiplications by 2 required to reach 8 from 1, i.e., if m is that number, we want an m such that: 2^m == **8**.
- This is exactly the definition of log(**8**) (log = logarithm base 2)
- Substitute 8 with **n** and you get (roughly) **m = log(n)**.

Insertion Sort

Consider the following sorting algorithm:

```
i = 1
while i < len(arr):
    j = i
    while j > 0 and arr[j] < arr[j - 1]:
        swap arr[j] and arr[j-1]
        j--
    i++
```

arr has been sorted!

What about worst-case complexity?

Insertion Sort

Consider the following sorting algorithm:

```
i = 1
while i < len(arr): ← This is repeated n times
    j = i
    while j > 0 and arr[j] < arr[j - 1]: ← At most n - 1 times
        swap arr[j] and arr[j-1]
        j--
    i++
```

arr has been sorted!

Worst-case complexity: O(n x (n-1)) = O(n^2)

Bogus – Bogus

Consider the following sorting algorithm:

```
while arr is not sorted:
    randomly shuffle arr
return arr
```

arr is sorted!

What about worst-case time complexity?

Bogus – Bogus

Consider the following sorting algorithm:

```
while arr is not sorted: ← There are n! different permutations
    randomly shuffle arr ← Each shuffling takes roughly O(n)
return arr
```

arr is sorted!

- In the worst case, arr will never be shuffled correctly, so infinite time!
- On average, O((n-1)n!) (n! = 1·2·3·…·n), so, much (much) slower than everything we have seen so far!

Homework 1

Look at the animated image on the right. You can observe, in action, a sorting algorithm called **Quicksort.**

- Using only the image and nothing else (no internet, no books, no LLMs, no entity that can somehow help you) try to describe the algorithm in plain english and / or pseudocode.
- Look around the web / books for Quicksort. How accurate is your description?
- Implement Quicksort using python and test it on some lists of numbers.

Share your responses at: v.markos@mc-class.gr

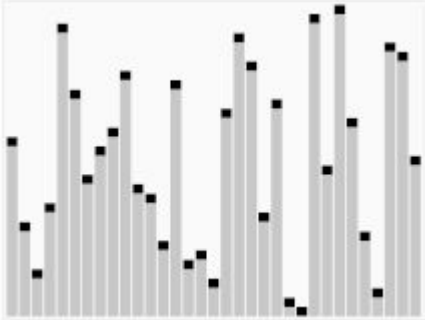


Image Source: https://en.wikipedia.org/wiki/Quicksort (do not open this!)

Fun Time!

Booker Paige, Again

- Open this week's tutorial found at './Labs/tutorial_01.pdf'.
- Follow the instructions therein.
- In case you have not completed your database design / normalisation from last week's tutorial, do it now!

Have Fun!

Any Questions?

Don’t forget to fill-in the questionnaire! (check right)

https://forms.gle/zezKMVzhaT4xyJgk8